

STACK

Q5)Check For Balanced Brackets Expression

DS	Session	Stack	Question Information	Level 1	Challenge 45		
Problem Description: Sajid was booking a tour package of IRCTC from Chennai to Delhi for his family. Two of the relatives was interested in joining to this tour. these two persons are studying engineering in computing technology, only one tickets are remaining in the IRCTC portal. So, Sajid decided to book one ticket for out of those persons also along with his family members. He wants to identify the one person out of these persons, he decided to conduct a technical task to identify the right person to travel. the task was that, Check for Balanced Brackets in an expression using Stack Can you help them to complete the task? Function Description • Declare a character stack S. • Now traverse the expression string exp. 1. If the current character is a starting bracket ('{' or '[' or '{') then push it to stack. 2. If the current character is a closing bracket (')' or ']' or '}') then pop from stack and if the popped character is the matching starting bracket then fine else brackets are not balanced. • After complete traversal, if there is some starting bracket left in stack then "not balanced" Input Format Single line represents only braces (both curly and square) Output Format If the given input balanced then print as Balanced [or] Not Balanced							
▼ Logical Test Cases <table><tr><td> Test Case 1 INPUT (STDIN) {()}[] EXPECTED OUTPUT Balanced </td><td> Test Case 2 INPUT (STDIN) {{({})}} EXPECTED OUTPUT Not Balanced </td></tr></table>						Test Case 1 INPUT (STDIN) {()}[] EXPECTED OUTPUT Balanced	Test Case 2 INPUT (STDIN) {{({})}} EXPECTED OUTPUT Not Balanced
Test Case 1 INPUT (STDIN) {()}[] EXPECTED OUTPUT Balanced	Test Case 2 INPUT (STDIN) {{({})}} EXPECTED OUTPUT Not Balanced						

CODE:

```
C main.c > main
1  #include <stdio.h>
2  #include <string.h>
3
4  int isMatching(char open, char close) {
5      if (open == '(' && close == ')') return 1;
6      if (open == '{' && close == '}') return 1;
7      if (open == '[' && close == ']') return 1;
8      return 0;
9  }
10
11 int main(void) {
12     char str[100005];
13     if (scanf(format: "%s", str) != 1) return 0;
14
15     char S[100005];
16     int top = -1;
17     int balanced = 1;
18
19     for (int i = 0; str[i] != '\0'; i++) {
20         char ch = str[i];
21         if (ch == '(' || ch == '{' || ch == '[') {
22             S[++top] = ch;
23         } else if (ch == ')' || ch == '}' || ch == ']') {
24             if (top == -1 || !isMatching(open: S[top--], close: ch)) {
25                 balanced = 0;
26                 break;
27             }
28         }
29     }
30
31     if (balanced && top == -1) {
32         printf(format: "Balanced\n");
33     } else {
34         printf(format: "Not Balanced\n");
35     }
36
37     return 0;
38 }
```

OUTPUT:

```
[suraj@suraj-83gs test]$ ./main
{()}[]
Balanced
[suraj@suraj-83gs test]$
```